

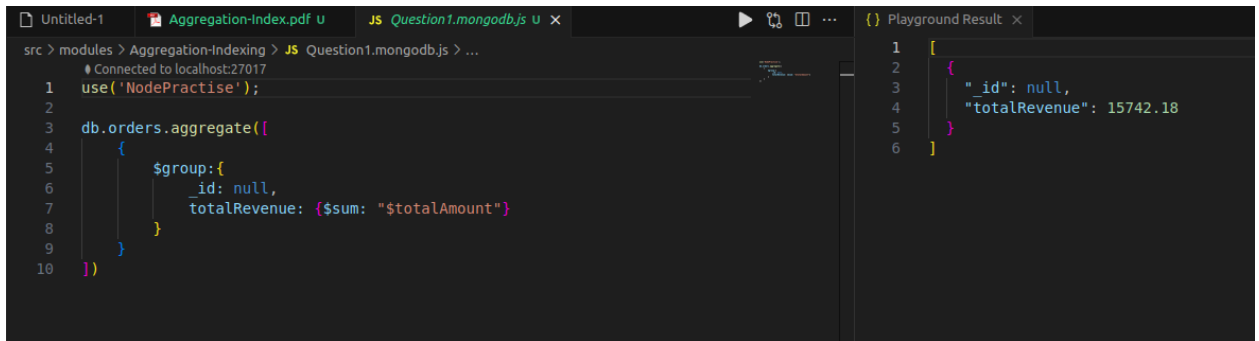
# Aggregation and Indexing Assignment

Sample Json-

```
{
  "_id": ObjectId,
  "orderId": String,
  "customerName": String,
  "orderDate": Date,
  "status": String, // e.g., "Pending", "Shipped", "Delivered"
  "items": [
    { "productName": String, "quantity": Number, "price": Number }
  ],
  "totalAmount": Number
}
```

Aggregation Queries Results( Also implemented on Mongo Compass )

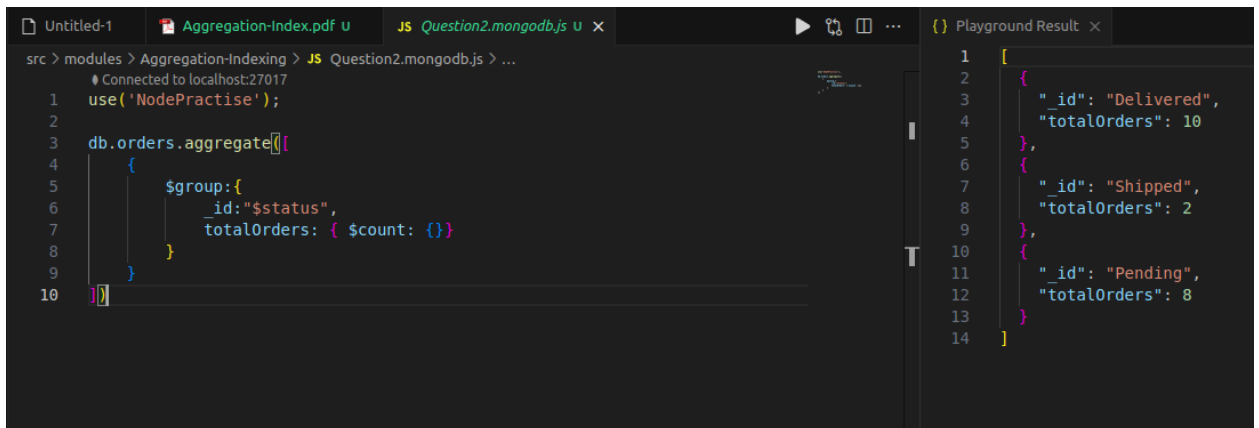
1.



```
src > modules > Aggregation-Indexing > JS Question1.mongodb.js > ...
# Connected to localhost:27017
1 use('NodePractise');
2
3 db.orders.aggregate([
4   {
5     $group:{
6       _id: null,
7       totalRevenue: {$sum: "$totalAmount"}
8     }
9   }
10 ])

Playground Result x
1 [
2   {
3     "_id": null,
4     "totalRevenue": 15742.18
5   }
6 ]
```

2



```
src > modules > Aggregation-Indexing > JS Question2.mongodb.js > ...
# Connected to localhost:27017
1 use('NodePractise');
2
3 db.orders.aggregate([
4   {
5     $group:{
6       _id:"$status",
7       totalOrders: { $count: {} }
8     }
9   }
10 ])

Playground Result x
1 [
2   {
3     "_id": "Delivered",
4     "totalOrders": 10
5   },
6   {
7     "_id": "Shipped",
8     "totalOrders": 2
9   },
10  {
11    "_id": "Pending",
12    "totalOrders": 8
13  }
14 ]
```

3

```

src > modules > Aggregation-Indexing > JS Question3.mongodb.js > ...
# Connected to localhost:27017
1 use('NodePractise');
2
3 db.orders.aggregate([
4   {
5     $group: {
6       _id: "$customerName",
7       totalSpent: {$sum: "$totalAmount"}
8     }
9   },
10  {
11    $sort: { totalSpent: -1}
12  },
13  {
14    $limit: 3
15  }
16 ],
17 )

```

```

1 [
2   {
3     "_id": "Antonia Hessel",
4     "totalSpent": 1608.88
5   },
6   {
7     "_id": "Edith Lesch",
8     "totalSpent": 1400.66
9   },
10  {
11    "_id": "Johanna Gulowski",
12    "totalSpent": 1307.64
13  }
14 ]

```

4

```

src > modules > Aggregation-Indexing > JS Question4.mongodb.js > ...
# Connected to localhost:27017
1 use('NodePractise');
2
3 db.orders.aggregate([
4   {
5     $group: {
6       _id: "$customerName",
7       averageTotal: {$avg: "$totalAmount"}
8     }
9   },
10  {
11    $sort: { averageTotal: -1}
12  },
13  {
14    $limit: 10
15  }
16 ],
17 )

```

```

1 [
2   {
3     "_id": "Elmer Koelpin",
4     "averageTotal": 508.13
5   },
6   {
7     "_id": "Horace Smith",
8     "averageTotal": 559.06
9   },
10  {
11    "_id": "Terence Gutkowski",
12    "averageTotal": 1182.65
13  },
14  {
15    "_id": "Antonia Hessel",
16    "averageTotal": 1608.88
17  },
18  {
19    "_id": "Angelica Cole",
20    "averageTotal": 1101.02
21  },
22  {
23    "_id": "Jenny Glover",
24    "averageTotal": 287.82
25  },
26  {
27    "_id": "Alicia Connelly",
28    "averageTotal": 997.4
29  },
30  {
31    "_id": "Edith Lesch",
32    "averageTotal": 1400.66
33  },
34  {
35    "_id": "Elisa Ritchie",
36    "averageTotal": 1275.84
37  },
38  {
39    "_id": "Johanna Gulowski",
40    "averageTotal": 1307.64
41  },
42  {
43    "_id": "Edith Lesch",
44    "averageTotal": 1400.66
45  },
46  {
47    "_id": "Antonia Hessel",
48    "averageTotal": 1608.88
49  },
50  {
51    "_id": "Elmer Koelpin",
52    "averageTotal": 508.13
53  },
54  {
55    "_id": "Horace Smith",
56    "averageTotal": 559.06
57  },
58  {
59    "_id": "Terence Gutkowski",
60    "averageTotal": 1182.65
61  },
62  {
63    "_id": "Antonia Hessel",
64    "averageTotal": 1608.88
65  },
66  {
67    "_id": "Angelica Cole",
68    "averageTotal": 1101.02
69  },
70  {
71    "_id": "Jenny Glover",
72    "averageTotal": 287.82
73  },
74  {
75    "_id": "Alicia Connelly",
76    "averageTotal": 997.4
77  },
78  {
79    "_id": "Edith Lesch",
80    "averageTotal": 1400.66
81  },
82  {
83    "_id": "Elisa Ritchie",
84    "averageTotal": 1275.84
85  },
86  {
87    "_id": "Johanna Gulowski",
88    "averageTotal": 1307.64
89  },
90  {
91    "_id": "Edith Lesch",
92    "averageTotal": 1400.66
93  },
94  {
95    "_id": "Antonia Hessel",
96    "averageTotal": 1608.88
97  },
98  {
99    "_id": "Elmer Koelpin",
100   "averageTotal": 508.13
101 }

```

5

```

src > modules > Aggregation-Indexing > JS Question5.mongodb.js > ...
# Connected to localhost:27017
1 use('NodePractise');
2
3 db.orders.aggregate([
4   {
5     $unwind: {
6       path: "$items",
7     }
8   },
9   {
10    $group: {
11      _id: "$items.productName",
12      quantity: { $sum: "$items.quantity" }
13    }
14  },
15  {
16    $match: {
17      quantity: { $gt: 10 }
18    }
19  }
20 ]
21 )

```

```

1 [
2   {
3     "_id": "Bespoke Rubber Tuna",
4     "quantity": 16
5   }
6 ]

```

6

```

src > modules > Aggregation-Indexing > JS Question6.mongodb.js > ...
# Connected to localhost:27017
1 use('NodePractise');
2
3 db.orders.aggregate([
4   {
5     $match: {
6       orderDate: {
7         $gte: new Date(new Date().setMonth(new Date().getMonth() - 6))
8       }
9     }
10  },
11  {
12    $group: {
13      _id: {
14        year: { $year: "$orderDate" },
15        month: { $month: "$orderDate" }
16      },
17      monthlyRevenue: { $sum: "$totalAmount" }
18    }
19  },
20  {
21    $sort: {
22      "_id.year": -1,
23      "_id.month": -1
24    }
25  }
26 ]
27 )

```

```

1 [
2   {
3     "_id": {
4       "year": 2025,
5       "month": 7
6     },
7     "monthlyRevenue": 6085.1
8   },
9   {
10    "_id": {
11      "year": 2025,
12      "month": 6
13    },
14    "monthlyRevenue": 3957.4900000000002
15  },
16  {
17    "_id": {
18      "year": 2025,
19      "month": 5
20    },
21    "monthlyRevenue": 1400.66
22  },
23  {
24    "_id": {
25      "year": 2025,
26      "month": 4
27    },
28    "monthlyRevenue": 2074.94
29  },
30  {
31    "_id": {
32      "year": 2025,
33      "month": 3
34    },
35    "monthlyRevenue": 1960.74
36  },
37  {
38    "_id": {
39      "year": 2025,
40      "month": 2
41    },
42    "monthlyRevenue": 263.25
43  }
44 ]

```

7

```

src > modules > Aggregation-Indexing > JS Question7.mongodb.js > $match > totalOrders > Sgt
1 use('NodePractise');
2
3 db.orders.aggregate([
4   {
5     $group: {
6       _id: "$customerName",
7       totalOrders: { $sum: 1 }
8     },
9   },
10  {
11    $match: {
12      totalOrders: { $gt: 0 }
13    },
14  },
15  {
16    $project: {
17      _id: 0,
18      customerName: "$_id",
19      totalOrders: 1
20    }
21  }
22 ])

```

```

1 {
2   {
3     "totalOrders": 1,
4     "customerName": "Edith Lesch"
5   },
6   {
7     "totalOrders": 1,
8     "customerName": "Ramon Durgan"
9   },
10  {
11    "totalOrders": 1,
12    "customerName": "Raymond Ratke"
13  },
14  {
15    "totalOrders": 1,
16    "customerName": "Louise Hansen"
17  },
18  {
19    "totalOrders": 1,
20    "customerName": "Gilbert Yundt"
21  },
22  {
23    "totalOrders": 1,
24    "customerName": "Mr. Abraham Kerluke"
25  },
26  {
27    "totalOrders": 1,
28    "customerName": "Elmer Koelpin"
29  },
30  {
31    "totalOrders": 1,
32    "customerName": "Horace Smith"
33  },
34  {
35    "totalOrders": 1,
36    "customerName": "Raquel Turcotte"
37  },

```

8

```

src > modules > Aggregation-Indexing > JS Question8.mongodb.js > ...
1 use('NodePractise')
2
3 db.orders.aggregate([
4   {
5     $unwind: {
6       path: "$items",
7     },
8   },
9   {
10    $group: {
11      _id: "$items.productName",
12    },
13  },
14  {
15    $project: {
16      _id: 0,
17      productName: "$_id",
18    }
19  }
20 ])

```

```

1 {
2   {
3     "productName": "Elegant Cotton Chair"
4   },
5   {
6     "productName": "Modern Steel Shoes"
7   },
8   {
9     "productName": "Soft Granite Soap"
10  },
11  {
12    "productName": "Rustic Bamboo Chair"
13  },
14  {
15    "productName": "Tasty Plastic Keyboard"
16  },
17  {
18    "productName": "Intelligent Cotton Fish"
19  },
20  {
21    "productName": "Fresh Aluminum Fish"
22  },
23  {
24    "productName": "Awesome Aluminum Chair"
25  },
26  {
27    "productName": "Recycled Metal Salad"
28  },
29  {
30    "productName": "Tasty Metal Cheese"
31  },
32  {
33    "productName": "Refined Rubber Car"
34  },
35  {
36    "productName": "Ergonomic Granite Mouse"
37  },
38  {
39    "productName": "Incredible Marble Ball"
40  },
41  {
42    "productName": "Elegant Metal Ball"
43  },

```

9

```
src > modules > Aggregation-Indexing > JS Question9.mongodb.js > ...
# Connected to localhost:27017
use('NodePractise')

1
2
3 db.orders.aggregate([
4   {
5     $match: {
6       status: "Delivered"
7     }
8   },
9   {
10    $group: {
11      _id: null,
12      revenue: {$sum: "$totalAmount"}
13    }
14  },
15  {
16    $project: {
17      _id: 0,
18      totalRevenue: "$revenue"
19    }
20  }
21 ])
```

```
1 [
2   {
3     "totalRevenue": 8375.74
4   }
5 ]
```

10

```
src > modules > Aggregation-Indexing > JS Question10.mongodb.js > ...
# Connected to localhost:27017
use('NodePractise')

1
2
3 db.orders.aggregate([
4   {
5     $unwind: {
6       path: "$items",
7     }
8   },
9   {
10    $group: {
11      _id: "$items.productName",
12      totalQuantity: {$sum: "$items.quantity"},
13      totalRevenue: {$sum: {$multiply: ["$items.price", "$items.quantity"]}}
14    }
15  }
16 ])
```

```
1 [
2   {
3     "_id": "Soft Ceramic Cheese",
4     "totalQuantity": 2,
5     "totalRevenue": 75.8
6   },
7   {
8     "_id": "Elegant Cotton Chair",
9     "totalQuantity": 2,
10    "totalRevenue": 193.38
11  },
12  {
13    "_id": "Modern Steel Shoes",
14    "totalQuantity": 1,
15    "totalRevenue": 72.55
16  },
17  {
18    "_id": "Soft Granite Soap",
19    "totalQuantity": 1,
20    "totalRevenue": 22.92
21  },
22  {
23    "_id": "Modern Bronze Chicken",
24    "totalQuantity": 3,
25    "totalRevenue": 131.67000000000002
26  },
27  {
28    "_id": "Rustic Bamboo Chair",
29    "totalQuantity": 1,
30    "totalRevenue": 41.39
31  },
32  {
33    "_id": "Fresh Aluminum Fish",
34    "totalQuantity": 9,
35    "totalRevenue": 149.85
36  },
37  {
38    "_id": "Awesome Aluminum Chair",
39    "totalQuantity": 6,
40    "totalRevenue": 519.54
41  },
42  {
43    "_id": "Recycled Metal Salad",
44    "totalQuantity": 1,
45    "totalRevenue": 12.99
46  }
47 ]
```

Index Queries:

1.

```
JS Indexing1.mongodbs.js x
src > modules > Aggregation-Indexing > JS Indexing1.mongodbs.js
  Connected to localhost:27017
1 use('NodePractise')
2 db.orders.getIndexes();

Playground Result x
1 [
2   {
3     "v": 2,
4     "key": {
5       "_id": 1
6     },
7     "name": "_id_"
8   }
9 ]
```

2. Before-

```
JS Indexing2.mongodbs.js M x TS script.ts
src > modules > Aggregation-Indexing > JS Indexing2.mongodbs.js > ...
  Connected to localhost:27017
1 use('NodePractise')
2 db.orders.find({customerName: "Maryann Nikolaus"}).explain(["executionStats"])

Playground Result x
1 {
2   "explainVersion": "1",
3   "queryPlanner": {
4     "namespace": "NodePractise.orders",
5     "parsedQuery": {
6       "customerName": {
7         "$eq": "Maryann Nikolaus"
8       }
9     },
10    "indexFilterSet": false,
11    "queryHash": "17B49A18",
12    "planCacheShapeHash": "17B49A18",
13    "planCacheKey": "0557E263",
14    "optimizationTimeMillis": 0,
15    "maxIndexedOrSolutionsReached": false,
16    "maxIndexedAndSolutionsReached": false,
17    "maxScansToExplodeReached": false,
18    "prunedSimilarIndexes": false,
19    "winningPlan": {
20      "isCached": false,
21      "stage": "FETCH",
22      "inputStage": {
23        "stage": "IXSCAN",
24        "keyPattern": {
25          "customerName": 1
26        },
27        "indexName": "customerName_1",
28        "isMultiKey": false,
29        "multiKeyPaths": {
30          "customerName": []
31        },
32        "isUnique": false,
33        "isSparse": false,
34        "isPartial": false,
35        "indexVersion": 2,
36        "direction": "forward",
37        "indexBounds": {
38          "customerName": [
39            ["\"Maryann Nikolaus\"", "\"Maryann Nikolaus\""]
40          ]
41        }
42      },
43    },
44    "rejectedPlans": []
45  },
46  "executionStats": {
47    "executionSuccess": true,
48    "nReturned": 1,
49    "executionTimeMillis": 6,
```

After-

```
JS Indexing2.mongodb.js M x TS script.js ... {} Playground Result: x {} ...
src > modules > Aggregation-Indexing > JS Indexing2.mongodb.js > ...
# Connected to localhost:27017
1 NodePractise
2 | ders.find({ customerName: "Maryann Nikolaus" }).explain("executionStats")

1 {
2   "explainVersion": "1",
3   "queryPlanner": {
4     "namespace": "NodePractise.orders",
5     "parsedQuery": {
6       "customerName": {
7         "$eq": "Maryann Nikolaus"
8       }
9     },
10    "indexFilterSet": false,
11    "queryHash": "17849A18",
12    "planCacheShapeHash": "17849A18",
13    "planCacheKey": "56634808",
14    "optimizationTimeMillis": 0,
15    "maxIndexedOrSolutionsReached": false,
16    "maxIndexedAndSolutionsReached": false,
17    "maxScansToExplodeReached": false,
18    "prunedSimilarIndexes": false,
19    "winningPlan": {
20      "isCached": false,
21      "stage": "COLLSCAN",
22      "filter": {
23        "customerName": {
24          "$eq": "Maryann Nikolaus"
25        }
26      },
27      "direction": "forward"
28    },
29    "rejectedPlans": []
30  },
31  "executionStats": {
32    "executionSuccess": true,
33    "nReturned": 1,
34    "executionTimeMillis": 0,
35    "totalKeysExamined": 0,
36    "totalDocsExamined": 20,
37    "executionStages": {
38      "isCached": false,
39      "stage": "COLLSCAN",
40      "filter": {
41        "customerName": {
42          "$eq": "Maryann Nikolaus"
43        }
44      },
45      "nReturned": 1,
46      "executionTimeMillisEstimate": 0,
47      "works": 21,
48      "advanced": 1,
49      "needTime": 19,
50      "needYield": 0,
```

3

Before-

```
JS Indexing3.mongodb.js M x JS Indexing3.mongodb.js M x ... {} Playground Result: x {} ...
src > modules > Aggregation-Indexing > JS Indexing3.mongodb.js > ...
# Connected to localhost:27017
1 use('NodePractise');
2
3 | db.orders.find({ status: "Delivered" }).explain("executionStats")

1 {
2   "explainVersion": "1",
3   "queryPlanner": {
4     "namespace": "NodePractise.orders",
5     "parsedQuery": {
6       "status": {
7         "$eq": "Delivered"
8       }
9     },
10    "indexFilterSet": false,
11    "queryHash": "506543D9",
12    "planCacheShapeHash": "506543D9",
13    "planCacheKey": "405CB45D",
14    "optimizationTimeMillis": 0,
15    "maxIndexedOrSolutionsReached": false,
16    "maxIndexedAndSolutionsReached": false,
17    "maxScansToExplodeReached": false,
18    "prunedSimilarIndexes": false,
19    "winningPlan": {
20      "isCached": false,
21      "stage": "COLLSCAN",
22      "filter": {
23        "status": {
24          "$eq": "Delivered"
25        }
26      },
27      "direction": "forward"
28    },
29    "rejectedPlans": []
30  },
31  "executionStats": {
32    "executionSuccess": true,
33    "nReturned": 7,
34    "executionTimeMillis": 0,
35    "totalKeysExamined": 0,
36    "totalDocsExamined": 20,
37    "executionStages": {
38      "isCached": false,
39      "stage": "COLLSCAN",
40      "filter": {
41        "status": {
42          "$eq": "Delivered"
43        }
44      },
45      "nReturned": 7,
46      "executionTimeMillisEstimate": 0,
47      "works": 21,
48      "advanced": 7,
49      "needTime": 13,
50      "needYield": 0,
51      "saveState": 0,
```

## After-

```
JS Indexing2.mongodb.js M JS Indexing3.mongodb.js M ... {} Playground Result x
src > modules > Aggregation-Indexing > JS Indexing3.mongodb.js > ...
  Connected to localhost:27017
1 use('NodePractise');
2
3 db.orders.find(
4   { status: "Delivered", orderDate: { $gte: ISODate("2024-01-01") } }
5   ).explain("executionStats")
6

1 {
2   "explainVersion": "1",
3   "queryPlanner": {
4     "namespace": "NodePractise.orders",
5     "parsedQuery": {
6       "$and": [
7         {
8           "status": {
9             "$eq": "Delivered"
10          },
11        },
12        {
13          "orderDate": {
14            "$gte": {
15              "$date": "2024-01-01T00:00:00Z"
16            }
17          }
18        }
19      ]
20    },
21    "indexFilterSet": false,
22    "queryHash": "888CD4EF",
23    "planCacheShapeHash": "888CD4EF",
24    "planCacheKey": "ABF87C76",
25    "optimizationTimeMillis": 0,
26    "maxIndexedOrSolutionsReached": false,
27    "maxIndexedAndSolutionsReached": false,
28    "maxScansToExplodeReached": false,
29    "prunedSimilarIndexes": false,
30    "winningPlan": {
31      "isCached": false,
32      "stage": "FETCH",
33      "inputStage": {
34        "stage": "IXSCAN",
35        "keyPattern": {
36          "status": 1,
37          "orderDate": -1
38        },
39        "indexName": "status_1_orderDate_-1",
40        "isMultiKey": false,
41        "multiKeyPaths": {
42          "status": [],
43          "orderDate": []
44        },
45        "isUnique": false,
46        "isSparse": false,
47        "isPartial": false,
48        "indexVersion": 2,
49        "direction": "forward",
50        "indexBounds": {
51          "status": [
```

4.

```
JS Indexing2.mongodb.js M JS Indexing3.mongodb.js M JS Indexing4.mongodb.js M ... {} Playground Result x
src > modules > Aggregation-Indexing > JS Indexing4.mongodb.js > ...
  Connected to localhost:27017
1 use('NodePractise')
2
3 db.orders.createIndex({ "items.productName": "text" })
4
5
6

1 items.productName_text
```



```
JS Indexing2.mongodb.js M JS Indexing3.mongodb.js M JS Indexing4.mongodb.js M X {} Playground Result x
src > modules > Aggregation-Indexing > JS Indexing4.mongodb.js > ...
1 use("NodePractise")
2 db.orders.find(
3   { $text: { $search: "Laptop" } }
4 ).explain("executionStats")
5

1 {
2   "explainVersion": "1",
3   "queryPlanner": {
4     "namespace": "NodePractise.orders",
5     "parsedQuery": {
6       "$text": {
7         "$search": "Laptop",
8         "$language": "english",
9         "$caseSensitive": false,
10        "$diacriticSensitive": false
11      }
12    },
13    "indexFilterSet": false,
14    "queryHash": "CF6F4CEE",
15    "planCacheShapeHash": "CF6F4CEE",
16    "planCacheKey": "08852285",
17    "optimizationTimeMillis": 0,
18    "maxIndexedOrSolutionsReached": false,
19    "maxIndexedAndSolutionsReached": false,
20    "maxScansToExplodeReached": false,
21    "prunedSimilarIndexes": false,
22    "winningPlan": {
23      "isCached": false,
24      "stage": "TEXT_MATCH",
25      "indexPrefix": {},
26      "indexName": "items.productName_text",
27      "parsedTextQuery": {
28        "terms": [
29          "Laptop"
30        ],
31        "negatedTerms": [],
32        "phrases": [],
33        "negatedPhrases": []
34      },
35      "textIndexVersion": 3,
36      "inputStage": {
37        "stage": "FETCH",
38        "inputStage": {
39          "stage": "IXSCAN",
40          "keyPattern": {
41            "_fts": "text",
42            "_ftsx": 1
43          },
44          "indexName": "items.productName_text",
45          "isMultiKey": true,
46          "isUnique": false,
47          "isSparse": false,
48          "isPartial": false,
49          "indexVersion": 2,
50          "direction": "backward",
51          "indexBounds": {
```

5.

## Before-

```
src > modules > Aggregation-Indexing > JS Indexing5.mongodb.js > customerName
  Connected to localhost:27017
1 use('NodePractise');
2
3 db.orders.find({customerName: "Refined Bronze Bike"}).explain("executionS
4

1 {
2   "explainVersion": "1",
3   "queryPlanner": {
4     "namespace": "NodePractise.orders",
5     "parsedQuery": {
6       "customerName": {
7         "$eq": "Refined Bronze Bike"
8       }
9     },
10    "indexFilterSet": false,
11    "queryHash": "17849A18",
12    "planCacheShapeHash": "17849A18",
13    "planCacheKey": "0557E263",
14    "optimizationTimeMillis": 0,
15    "maxIndexedOrSolutionsReached": false,
16    "maxIndexedAndSolutionsReached": false,
17    "maxScansToExplodeReached": false,
18    "prunedSimilarIndexes": false,
19    "winningPlan": {
20      "isCached": false,
21      "stage": "FETCH",
22      "inputStage": {
23        "stage": "IXSCAN",
24        "keyPattern": {
25          "customerName": 1
26        },
27        "indexName": "customerName_1",
28        "isMultiKey": false,
29        "multiKeyPaths": {
30          "customerName": []
31        },
32        "isUnique": false,
33        "isSparse": false,
34        "isPartial": false,
35        "indexVersion": 2,
36        "direction": "forward",
37        "indexBounds": {
38          "customerName": [
39            ["Refined Bronze Bike", "Refined Bronze Bike"]
40          ]
41        }
42      },
43    },
44    "rejectedPlans": []
45  },
46  "executionStats": {
47    "executionSuccess": true,
48    "nReturned": 0,
49    "executionTimeMillis": 0,
50    "totalKeysExamined": 0,
51    "totalDocsExamined": 0
```

## After-

```
src > modules > Aggregation-Indexing > JS Indexing5.mongodb.js > customerName
  * Connected to localhost:27017
1 use('NodePractise');
2 db.orders.find({customerName: "Refined Bronze Bike"}).explain("executionStats");
3
4
```

```
1 {
2   "explainVersion": "1",
3   "queryPlanner": {
4     "namespace": "NodePractise.orders",
5     "parsedQuery": {
6       "customerName": {
7         "$eq": "Refined Bronze Bike"
8       }
9     },
10    "indexFilterSet": false,
11    "queryHash": "17849A18",
12    "planCacheShapeHash": "17849A18",
13    "planCacheKey": "56634808",
14    "optimizationTimeMillis": 0,
15    "maxIndexedOrSolutionsReached": false,
16    "maxIndexedAndSolutionsReached": false,
17    "maxScansToExplodeReached": false,
18    "prunedSimilarIndexes": false,
19    "winningPlan": {
20      "isCached": false,
21      "stage": "COLLSCAN",
22      "filter": {
23        "customerName": {
24          "$eq": "Refined Bronze Bike"
25        }
26      },
27      "direction": "forward"
28    },
29    "rejectedPlans": []
30  },
31  "executionStats": {
32    "executionSuccess": true,
33    "nReturned": 0,
34    "executionTimeMillis": 0,
35    "totalKeysExamined": 0,
36    "totalDocsExamined": 20,
37    "executionStages": {
38      "isCached": false,
39      "stage": "COLLSCAN",
40      "filter": {
41        "customerName": {
42          "$eq": "Refined Bronze Bike"
43        }
44      },
45      "nReturned": 0,
46      "executionTimeMillisEstimate": 0,
47      "works": 21,
48      "advanced": 0,
49      "needTime": 20,
50      "needYield": 0,
51      "saveState": 0
```